

База знаний Senior Android Engineer — 2026

Самоподготовка и калибровка уровня. стек: Jetpack Compose, Coroutines + Flow, MVI (Actor/Reducer/NewsPublisher), Dagger 2 (manual injection), кастомная Fragment-навигация с Compose внутри, Room + DataStore, Retrofit/OkHttp, JUnit5 + MockK + Turbine, Gradle Version Catalogs + R8 + Baseline Profiles, Timber. Базовые OOP/SOLID/Git предполагаются известными и не объясняются.

Тема 1. Kotlin & Coroutines internals

Теория

suspend-функция компилируется в CPS (continuation-passing style): компилятор добавляет скрытый параметр `Continuation`, а тело режется на state machine, где точки suspend — это метки состояний; `Continuation.resumeWith` продвигает машину дальше. Структурированная конкурентность означает, что каждый билдер привязан к `Job` в `CoroutineContext`, дочерние Job образуют дерево, и родитель не завершится, пока не завершатся дети. Отмена кооперативна: `cancel()` лишь переводит Job в Cancelling и бросает `CancellationException` на ближайшем suspend-вызове, проверяющем `isActive`; CPU-bound цикл без suspend не отменится, пока вы не вставите `ensureActive()`/`yield()`. Ключевое разделение обработки ошибок: `try/catch` ловит синхронно вокруг suspend-вызова; `CoroutineExceptionHandler` срабатывает только на корневой coroutine скоупа (для дочерних и для `launch` он игнорируется, а у `async` исключение всплывает на `await()`); `SupervisorJob`/`supervisorScope` изолируют падение детей, чтобы крах одного не отменял остальных.

Вопрос 1.1 — Чем отличаются try/catch, CoroutineExceptionHandler и SupervisorJob?

Junior. try/catch ловит исключения, CoroutineExceptionHandler — это глобальный обработчик для coroutine, а SupervisorJob не даёт упасть всему скоупу. Все три про обработку ошибок.

Middle. try/catch оборачивает конкретный suspend-вызов. CoroutineExceptionHandler ставится в контекст скоупа и логирует необработанные исключения. SupervisorJob нужен, чтобы падение одного ребёнка не отменяло сиблингов; с обычным Job падение любого ребёнка отменяет родителя и всех остальных. У `async` исключение нужно ловить на `await()`.

Senior. Это три разных уровня. `try/catch` — синхронный механизм; он не поймает исключение из дочернего `launch`, потому что оно распространяется вверх по дереву `Job`, а не по стеку вызовов. `CoroutineExceptionHandler` — «последний рубеж» и работает ТОЛЬКО на корневой `coroutine` скоупа: для дочерних `coroutine` он молча игнорируется, потому что они делегируют исключение родителю; для `async` он бесполезен, так как исключение упаковано в `Deferred` и всплывает на `await`. `SupervisorJob` меняет политику распространения: дети не отменяют родителя, но важно, что супервизия действует на один уровень — вложенный обычный `coroutineScope` внутри `supervisorScope` восстановит обычную семантику. На практике: `handler` ставим на корне (например, в `viewModelScope` через `CoroutineExceptionHandler`), а изоляцию независимых задач даём через `supervisorScope` или `SupervisorJob`, при этом `CancellationException` никогда не отдаём в `handler` — её нужно пробрасывать дальше.

Почему это важно. Вопрос отделяет тех, кто читал, от тех, кто отлаживал падения в проде. Незнание, что `handler` не ловит детей и `async`, приводит к «проглоченным» крашам и к тому, что один сбойный запрос гасит весь экран. Ответ показывает, понимает ли человек, что ошибки в `coroutines` идут по дереву `Job`, а не по `call stack`.

Вопрос 1.2 — Как работает кооперативная отмена и где она ломается?

Junior. Надо вызвать `cancel()`, и `coroutine` остановится. `delay()` умеет отменяться.

Middle. Отмена кооперативна: `suspend`-функции из `kotlinx.coroutines` проверяют отмену и бросают `CancellationException`. Если внутри тяжёлый цикл без `suspend`, его надо проверять через `isActive`/`ensureActive()`/`yield()`, иначе он доработает до конца. В `finally` для `suspend`-очистки нужен `withContext(NonCancellable)`.

Senior. `cancel()` лишь переводит `Job` в состояние `Cancelling` и взводит флаг; реальная остановка происходит на первой `cancellation point`. Поэтому CPU-bound код невидим для отмены — нужен ручной `ensureActive()`. Важные грабли: проглатывание `CancellationException` в широком `catch (e: Exception)` ломает структурированную конкурентность, так как отмена перестаёт распространяться — её всегда надо рефрейзить/пробрасывать. В `finally` обычные `suspend`-вызовы уже отменены, поэтому закрытие ресурсов с `suspend` оборачиваем в `NonCancellable`. И отмена идёт вниз по дереву: отменив родителя, мы отменяем всех детей, но родитель в `join` дожждётся их фактического завершения.

Почему это важно. Утечки и «зомби-задачи» почти всегда от непонимания, что отмена не мгновенная и кооперативная. Ответ про `catch (Exception)`, глотающий `CancellationException`, — маркер реального опыта дебага ANR/утечек.

Вопрос 1.3 — Что такое CPS и зачем это знать?

Junior. `suspend`-функции компилятор как-то превращает в `callbacks`, чтобы не блокировать поток.

Middle. Компилятор добавляет параметр `Continuation` и превращает функцию в `state machine`: каждая точка `suspend` — состояние, между ними код выполняется синхронно. Возврат — либо значение, либо маркер `COROUTINE_SUSPENDED`.

Senior. Понимание CPS объясняет реальные вещи: почему `suspend`-функция «бесплатна» по потокам (нет блокировки, есть приостановка с сохранением состояния в объекте `Continuation` на куче); почему стектрейсы рваные (нужен `debug`-режим `coroutines/(-ea)`); почему `withContext` дешёв, но не нулевой — это смена `dispatcher` и `resume` на другом потоке. Знание, что состояние локальных переменных уезжает в поля сгенерированного класса, помогает понимать, что захват больших объектов в долгой `coroutine` = их удержание в памяти всё время приостановки.

Почему это важно. Это водораздел между «умею вызывать `suspend`» и «понимаю, как это выполняется». Влияет на отладку, на профилирование памяти и на корректные ментальные модели `dispatcher`'ов.

Тема 2. Flow / StateFlow / SharedFlow

Теория

Flow холодный: код продьюсера запускается заново на каждый `collect`, контекст исполнения определяется потребителем (context preservation), а сменить `upstream`-контекст можно только через `flowOn`, который влияет на операторы ВЫШЕ по цепочке. StateFlow и SharedFlow горячие: живут независимо от коллекторов. StateFlow всегда имеет значение, `conflate`'ит (теряет промежуточные при быстрых эмиссиях) и дедуплицирует через `equals()` — два равных значения подряд не доставляются, что критично для `data`-классов. SharedFlow гибче: `replay`, `extraBufferCapacity`, `onBufferOverflow` (`SUSPEND/DROP_OLDEST/DROP_LATEST`) и без дедупликации. `stateIn`/`shareIn` превращают холодный Flow в горячий с политикой `SharingStarted`: `Eagerly` (сразу и навсегда), `Lazily` (с первого подписчика навсегда), `WhileSubscribed(stopTimeoutMillis, replayExpirationMillis)`. Каноничный `WhileSubscribed(5000)` держит `upstream` живым 5 секунд после ухода последнего коллектора, переживая `configuration change` без рестарта дорогого `upstream`; `replayExpirationMillis` по умолчанию `Long.MAX_VALUE` (кэш не сбрасывается при остановке).

Вопрос 2.1 — В чём разница cold Flow и hot StateFlow/SharedFlow и когда что?

Junior. Flow холодный — стартует на каждый `collect`, StateFlow всегда хранит значение, SharedFlow умеет несколько подписчиков. StateFlow для UI-состояния.

Middle. Cold Flow пересоздаёт продьюсера на каждого коллектора, поэтому два `collect` = два сетевых запроса. Hot-потоки эмитят независимо. StateFlow — для состояния экрана (всегда есть `value`, дедуп по `equals`). SharedFlow с `replay=0` — для событий. Холодный делаем горячим через `stateIn`/`shareIn` с правильным `scope`.

Senior. Главное — управление дороговизной и жизненным циклом. Холодный Flow дешёв и самодостаточен, но повторное подключение перезапускает upstream; чтобы мультикастить дорогой источник (БД, сокет), берём `shareIn/stateIn`. Выбор `SharingStarted` — это trade-off: `Eagerly` греет ресурс впустую; `WhileSubscribed(5000)` — золотой стандарт для ViewModel, переживает rotation, но «магическое число» не панацея — при долгом уходе с экрана (>5с) и возврате upstream перезапустится; для one-shot загрузок это лишний рефетч (отсюда кастомные `OnetimeWhileSubscribed`). StateFlow conflate'ит и дедупит по equals — если эмитить структурно равные объекты, коллектор их не увидит, а если изменять mutable-поле без `copy()`, новое значение «проглотится». SharedFlow требует осознанного `onBufferOverflow`: SUSPEND может незаметно затормозить эмиттер.

Почему это важно. Это сердце современного Android data flow. Ответ показывает, понимает ли человек, что неверный `SharingStarted` или эмиссия mutable-стейта = либо двойные запросы, либо «залипший» UI. Знание дефолта `replayExpiration` и семантики 5000 — маркер глубины.

Вопрос 2.2 — Что делают flowOn, buffer, conflate и как они влияют на backpressure?

Junior. flowOn меняет поток, buffer добавляет буфер, conflate оставляет последнее значение.

Middle. flowOn задаёт dispatcher для upstream-операторов (всё, что выше по цепочке), не трогая downstream. buffer развязывает продюсера и потребителя, давая им работать параллельно. conflate выкидывает промежуточные значения, если потребитель не успевает. В отличие от Rx, backpressure встроен через suspend.

Senior. Backpressure в Flow реализован суспендом: по умолчанию продюсер ждёт потребителя (rendezvous). `flowOn` создаёт channel-границу и меняет контекст ровно для операторов выше — частая ошибка ставить его в конце цепочки и ждать эффекта на collect. `buffer(capacity, onBufferOverflow)` — это явный канал между стадиями; `conflate()` = `buffer(CONFLATED)`; `collectLatest` отменяет недоработанную обработку при новом значении. Тонкость: несколько `flowOn` создают несколько каналов (накладные расходы), а `buffer` склеивается с соседним `flowOn`. Для UI обычно `conflate/collectLatest`, для аналитики — буфер без потерь.

Почему это важно. Отделяет тех, кто копирастит `flowOn(IO)`, от тех, кто понимает границы контекста и стоимость каналов. В проде это разница между плавным UI и дропнутыми/задвоенными событиями.

Вопрос 2.3 — Как мигрировать Rx-цепочку на Flow и тестировать через Turbine?

Junior. Observable → Flow, Single → suspend-функция, Subject → SharedFlow. Тестируем, собирая значения в список.

Middle. `Observable/Flowable` → `Flow`; `Single/Maybe` → `suspend-функция` (или `Flow` с одним элементом); `Completable` → `suspend` `Unit`; `BehaviorSubject` → `MutableStateFlow`; `PublishSubject` → `MutableSharedFlow(replay=0)`. `flatMap` → `flatMapMerge`, `concatMap` → `flatMapConcat`, `switchMap` → `flatMapLatest`. Turbine: `flow.test { assertEquals(x, awaitItem()); awaitComplete() }`.

Senior. Маппинг операторов прямой, но семантические ловушки важнее. Rx-схемы по умолчанию многопоточны через `subscribeOn/observeOn`, Flow — context preservation + `flowOn`, поэтому слепой перенос ломает потоки. Rx-`Subject` горячий и без backpressure (`PublishSubject` теряет, если нет подписчиков) — для эмиттеров событий важно подобрать `extraBufferCapacity`/`onBufferOverflow`, чтобы не словить SUSPEND или потерю. Обработка ошибок: `onErrorReturn` → `catch`, но `catch` ловит только upstream и не сработает на исключения downstream — частая ошибка. Turbine незаменим для горячих потоков: он форсит проверку, что НЕ осталось неконсумированных эмиссий (`ensureAllEventsConsumed`), чего ручной сбор в список не даёт; для `StateFlow` помним про начальное значение и conflation.

Почему это важно. Реальные команды мигрируют Rx инкрементально. Ответ выявляет, понимает ли человек, что Rx и Flow различаются не синтаксисом, а моделью многопоточности и backpressure — именно там возникают регрессии.

Тема 3. Compose: state, recomposition, performance

Теория

Recomposition пропускается (skip), если все параметры composable «не изменились»; что значит «изменились» — зависит от stability. С версии compiler 2.0.20 strong skipping mode включён по умолчанию и меняет правила. Точная семантика (подтверждено официальной документацией Android, design-doc JetBrains и статьёй Ben Trengrove на Android Developers Medium — все три источника согласованы): **unstable-параметры сравниваются по instance equality (`===`)**, **stable-параметры — по object equality (`Object.equals()`)**. То есть при strong skipping даже composable с unstable-параметрами становится skipable, но новый структурно-равный экземпляр unstable-типа (новый List, `copy()` data-класса, DTO из Room, переэмит `StateFlow`) считается изменившимся по `===` и форсит recomposition. Лямбды компилятор всегда считает stable и при strong skipping авто-оборачивает в `remember`, keyed по захватам (причём ключи сравниваются по тем же правилам: unstable-ключи по `===`, stable — по equals). `@Stable`/`@Immutable` по-прежнему ценны: согласно официальной документации, «если вы хотите, чтобы объект сравнивался по object equality вместо instance equality, продолжайте аннотировать класс через `@Stable`» — это нужно для типов, чьи экземпляры пересоздаются с равным контентом (классический случай — Room/StateFlow, отдающие новый список при изменении одного элемента), для типов из внешних модулей (по умолчанию unstable) и при дорогом `equals()` — но избыточное аннотирование типов с тяжёлым `equals()` и многими параметрами может стоить дороже дешёвой recomposition.

Вопрос 3.1 — Что такое `stability`, и изменил ли `strong skipping` смысл `@Stable`/`@Immutable`?

Junior. `@Stable`/`@Immutable` говорят Compose, что тип стабильный, и тогда `composable` можно скипать. `List` нестабилен, его лучше заменить на `ImmutableList`.

Middle. Compose скипает `composable`, если параметры стабильны и не изменились. Нестабильные типы (`var`-поля, интерфейсы, типы из других модулей, `List` как реализация) ломают `skip`. Раньше один нестабильный параметр делал всю функцию непрорускаемой, поэтому навешивали аннотации и брали `kotlinx.collections.immutable`.

Senior. Strong skipping (дефолт с 2.0.20) сделал большинство `@Stable`/`@Immutable` ненужными: теперь `composable` с `unstable`-параметрами тоже `skippable`, просто `unstable`-параметры сравниваются по `===`, а `stable` — по `equals()`.

Практический вывод: аннотации нужны там, где экземпляр пересоздаётся, но контентно равен — классический случай `Room`/`StateFlow`, отдающих новый список при изменении одного элемента: без стабильности `===` всегда даёт «изменилось». Тогда `@Immutable`/`@Stable` переводит сравнение на `equals()` и возвращает `skip`. Но это не бесплатно: для типов с большим числом параметров и дорогим `equals()` стоимость проверки может превысить стоимость дешёвой `recomposition`. Главная ловушка 2026: чтение `compiler`-метрик и паническое аннотирование всего «`unstable`» — под `strong skipping` большинство таких флагов безвредны, реальные узкие места обычно в другом (пропущенный `derivedStateOf`, чтение `state` слишком высоко).

Почему это важно. Это прямой маркер актуальности знаний. Кандидат, который в 2026 всё ещё «аннотирует каждый `data class`», отстал; senior знает, что `strong skipping` сместил проблему с «сделать `skippable`» на «сделать сравнимым по контенту, где это окупается».

Вопрос 3.2 — Назовите `side-effect API` и типичные ловушки.

Junior. `LaunchedEffect` для запуска `suspend` при входе в композицию, `DisposableEffect` для очистки, есть ещё `remember`.

Middle. `LaunchedEffect(key)` запускает `coroutine`, перезапускается при смене ключа. `DisposableEffect` — для подписок с `onDispose`. `derivedStateOf` — для вычисляемого `state`, чтобы не пересчитывать на каждый `recomposition`. `rememberUpdatedState` — чтобы `long-lived effect` видел свежее значение без перезапуска. `produceState` — мост из `non-Compose` источника в `State`. Частая ошибка — `LaunchedEffect(Unit)`, который не реагирует на смену параметра.

Senior. Ключевое — правильный выбор ключа и понимание, что есть «эффект перезапуска» против «свежего чтения».

`LaunchedEffect(key)`: ключ должен отражать, при каком изменении эффект надо рестартовать; `Unit` оправдан только для строго одноразового. `rememberUpdatedState` — когда эффект долгоживущий (таймер, listener), но колбэк должен видеть актуальное значение без рестарта. `derivedStateOf` критичен, когда часто меняющийся state (scroll offset) преобразуется в редко меняющийся булев (`isScrolled`) — иначе recomposition на каждый пиксель; и наоборот, для дешёвых вычислений `derivedStateOf` — лишний оверхед. `snapshotFlow` — мост из State в Flow. Ловушки: side-effect прямо в теле composable (не в effect-блоке); пропущенный ключ; неснятая подписка из-за `LaunchedEffect` вместо `DisposableEffect`.

Почему это важно. Side-effects — главный источник багов в Compose (двойные запросы, утечки listener'ов, перезапуски анимаций). Ответ показывает, мыслит ли человек жизненным циклом композиции, а не «куда воткнуть запуск».

Вопрос 3.3 — Перечислите перф-ловушки recomposition и как их чинить.

Junior. Не делать тяжёлых вычислений в composable, выносить в remember. Поменьше лишних recomposition.

Middle. Читать state как можно ниже по дереву (state read scope), оборачивать дорогие вычисления в `remember`. В `LazyColumn` задавать стабильные `key`. Не создавать новые лямбды/объекты в горячем пути без необходимости. Использовать Layout Inspector для подсчёта recomposition.

Senior. Системно: (1) Deferred reads — читать state как можно позже, в идеале через lambda-modifier (`Modifier.offset { ... }`), `graphicsLayer { }`), чтобы изменение уходило в фазу layout/draw, минуя recomposition вообще; это самый недооценённый приём. (2) Reading state too high поднимает инвалидацию на большое поддерево — поднимайте state holder, но читайте значение низко. (3) Стабильные `key` в lazy-списках сохраняют identity элементов при перестановках. (4) При strong skipping лямбды авто-мемоизируются, НО только внутри `@Composable`-скоупа — лямбды в `items { }` (LazyListScope не композбл) не мемоизируются, их по-прежнему надо `remember`. (5) Метрики компилятора (`reportsDestination`) показывают skippable/restartable, но читать их надо с поправкой на strong skipping. Чинить начинаем с измерения (Layout Inspector recomposition counts, compiler reports), а не с догадок. [DEV Community](#)

Почему это важно. Перф в Compose — про понимание трёх фаз (composition/layout/draw) и про то, что лучшая recomposition — та, которой нет. Ответ про lambda-modifiers и deferred reads отделяет тех, кто реально оптимизировал скролл, от теоретиков.

Тема 4. MVI и однопоточный поток данных

Теория

MVI в стиле MVICore (Badoo/Bumble) — это конечный автомат `Feature<Wish, State, News>`. Поток строго однонаправленный: UI шлёт Wish/Action через единственную точку входа `acceptAction(action)`; Actor (тип `(State, Action) -> Flow<Effect>`) выполняет асинхронную работу и условную логику на основе текущего state, эмитя поток Effect; Reducer `((State, Effect) -> State)` — чистая синхронная функция без побочных эффектов, создающая новый immutable State; NewsPublisher `((Action, Effect, State) -> News?)` порождает one-shot события (toast, snackbar, навигация), которые НЕ должны жить в state. State отдаётся наружу как `StateFlow`, News — как `Flow`. Дополнительно: Bootstrapper (стартовые Action), PostProcessor (Action по результату редукции). Reducer обязан вызываться на одном потоке, поэтому результаты асинхронных задач Actor сводим на единый dispatcher. Различие state vs news решает проблему SingleLiveEvent: state — это «истина, которую можно перерисовать», news — «событие, которое случается один раз».

Вопрос 4.1 — Почему однонаправленный поток и чем Actor/Reducer/NewsPublisher лучше MVVM?

Junior. В MVI один поток данных и одно состояние, меньше путаницы. В MVVM состояние размазано по нескольким LiveData.

Middle. UDF означает единственный источник истины (State) и предсказуемые переходы. Reducer как чистая функция легко тестируется и воспроизводится. MVVM с несколькими LiveData допускает несогласованные комбинации полей; MVI собирает всё в один immutable State. News отдельно от State решает проблему повторного показа событий при rotation.

Senior. Главная ценность — это конечный автомат с явными, тестируемыми переходами и невозможностью «невалидных» комбинаций состояния. Разделение ролей даёт системные свойства: Reducer чист и синхронен → детерминирован, воспроизводим, тривиально тестируется и пригоден для time-travel debugging; Actor инкапсулирует все side-effects и единственный знает про асинхронность; NewsPublisher решает фундаментальную проблему «событие vs состояние» — навигацию/тосты нельзя класть в State, иначе они переиграются при пересоздании. Trade-offs честно: MVI многословнее (sealed-иерархии Wish/Effect/State/News), для тривиального экрана это оверинжиниринг — там уместен простой `ReducerFeature` или даже MVVM. MVI выигрывает на сложных экранах с гонками, валидацией по текущему state и множеством источников. Грабли: держать в State то, что должно быть News (двойная навигация); делать Reducer нечистым (теряется тестируемость); раздувать один Feature вместо композиции.

Почему это важно. Архитектурный вопрос на зрелость. «MVI всегда лучше» — junior-абсолютизм; senior говорит «зависит от сложности экрана и цены гонок» и точно знает, ЗАЧЕМ News отделены от State.

Вопрос 4.2 — Как обрабатывать side-effects и в чём разница State vs News?

Junior. Side-effects (сеть, БД) — в Actor. State — то, что на экране, News — события вроде тоста.

Middle. Actor делает запрос и эмитит Effect (Loading/Success/Error), Reducer превращает их в State. Ошибку, которую надо показать один раз, отдаём через News, а не кладём в State (иначе при rotation покажется снова). Навигация — тоже News.

Senior. Критерий разделения: можно ли это «перерисовать»? State — декларативное описание экрана, идемпотентно применяемое сколько угодно раз (loading-спиннер, данные, inline-ошибка, которая должна оставаться видимой). News — императивное одноразовое действие (snackbar, навигация, vibration), которое при повторном применении даст баг. Side-effects живут в Actor, потому что только он имеет доступ к state и асинхронности; Reducer обязан остаться чистым. Тонкость потоков: News как `Flow` нужно собирать с учётом lifecycle (`repeatOnLifecycle`), иначе при подписке после эмиссии событие потеряется — поэтому для News иногда берут буферизированный канал/SharedFlow с буфером, чтобы пережить кратковременную потерю подписчика, балансируя «потерять» против «продублировать».

Почему это важно. Путаница state/news — классический источник «двойной навигации» и «тост при каждом повороте». Ответ показывает, сталкивался ли человек с реальной болью SingleLiveEvent и понимает ли семантику доставки one-shot событий.

Вопрос 4.3 — Как тестировать MVI-машину состояний?

Junior. Тестируем Reducer: подаём State и Effect, проверяем новый State.

Middle. Reducer тестируется как чистая функция без моков. Actor тестируем с подменой зависимостей (MockK) и проверяем последовательность Effect через Turbine. Для Feature целиком шлём Wish и проверяем итоговый StateFlow и News.

Senior. Многоуровневая стратегия. Reducer — чистые table-driven юнит-тесты без coroutines вообще, максимально дёшево. Actor — с TestDispatcher и Turbine на `Flow<Effect>`, проверяя и порядок, и условную логику (например, что при `state.isLoading` повторный Wish даёт пустой поток). Feature целиком — интеграционный тест входа `acceptAction` → итоговый State и эмиссия News, через `runTest`, с инжектированным TestDispatcher, чтобы свести reducer на один поток. Turbine здесь обязателен: он проверяет, что не осталось лишних/потерянных эмиссий, и отлавливает дедупликацию StateFlow. Тестируем именно переходы автомата (последовательности состояний), а не отдельные значения — это и есть проверка инвариантов FSM.

Почему это важно. MVI продаётся как «легко тестируемая» архитектура; вопрос проверяет, умеет ли человек реализовать это преимущество, а не только декларировать. Знание про Turbine и сведение reducer на один dispatcher — признак практики.

Тема 5. Dagger 2 (manual injection) vs Hilt

Теория

Component — это граф, генерируемый из набора Module и `@Inject`-конструкторов; entry-point'ы (`inject(activity)`) определяют, что нужно построить. Предпочитайте `@Inject`-конструктор (Dagger сам знает, как создать) модулям с `@Provides` (нужны для интерфейсов, сторонних классов, конфигулируемых объектов). Scopes (`@Singleton`, кастомные `@ActivityScope`) — это не «глобальность», а привязка времени жизни к компоненту; один scope нельзя переиспользовать между предком и потомком. Subcomponents наследуют ВСЁ граф родителя (включая multibindings) и создают единое пространство ключей; component dependencies — явная связь, где родитель экспонирует только объявленные через factory-методы объекты, что позволяет раздельную/параллельную компиляцию, но НЕ пробрасывает multibindings. Multibinding (`@IntoSet`, `@IntoMap`) собирает вклады из разных модулей в Set/Map — основа плагиновой архитектуры (например, Map субкомпонентов или ViewModel-фабрик). Assisted injection (`@AssistedInject`/`@AssistedFactory`) — когда часть параметров приходит из графа, а часть в рантайме. Hilt — это надстройка с предопределёнными компонентами и кодогенерацией; снимает бойлерплейт, но навязывает стандартную иерархию.

Вопрос 5.1 — Когда модуль с @Provides, а когда @Inject-конструктор?

Junior. @Inject на конструкторе своих классов, @Provides в модуле для всего остального.

Middle. `@Inject constructor` — для своих классов, которые Dagger может создать сам. `@Provides`/`@Binds` — для интерфейсов (привязка реализации), сторонних классов без доступа к конструктору, и объектов, требующих настройки (Retrofit, OkHttp, Room). `@Binds` эффективнее `@Provides` для простой привязки интерфейс→реализация.

Senior. Правило: всё, что можешь, отдавай через `@Inject`-конструктор — это меньше кода и Dagger видит граф напрямую. Модули нужны для трёх случаев: интерфейс→реализация (`@Binds`, abstract, нулевой рантайм-оверхед против `@Provides`), сторонние/конфигулируемые объекты, и multibindings. Тонкость: `@Binds` обязан быть abstract и не может соседствовать с `@Provides` в одном не-companion модуле без статиков. Архитектурно в мультимодульном проекте интерфейсы выносят в api-модуль, реализации с `@Binds` — в impl-модуль, что даёт инверсию зависимостей на уровне Gradle. Грабли: лишние `@Provides` там, где хватило бы конструктора, раздувают граф и время сборки.

Почему это важно. Показывает гигиену графа. Senior минимизирует модули и понимает разницу `@Binds`/`@Provides` по стоимости и по тому, как это ложится на модуляризацию.

Вопрос 5.2 — Subcomponents против component dependencies?

Junior. Subcomponent вложен в родителя и видит его зависимости. Component dependency — когда один компонент зависит от другого.

Middle. Subcomponent автоматически наследует весь граф родителя, объявляется через `@Subcomponent` + установку в модуль родителя. Component dependency требует явно экспонировать нужные объекты factory-методами в родителе. Subcomponent проще для тесной связи (Activity внутри App), dependencies — для слабой связи между независимо собираемыми частями.

Senior. Это trade-off «удобство против изоляции и скорости сборки». Subcomponents пробрасывают ВСЁ, включая multibindings — единое пространство ключей, проще писать, но сложнее ограничить, что фича видит из другой фичи, и нет параллельной сборки (потомок должен генерироваться вместе с предком). Component dependencies дают чёрные ящики: родитель экспонирует только объявленное, можно собирать параллельно и переиспользовать в разных графах, но multibindings НЕ пробрасываются (известное ограничение) и нужно вручную прокидывать каждую зависимость через factory-метод, плюс entry-point методы мешают tree-shaking мёртвых биндингов. Практика: subcomponents для внутренней иерархии (App→Activity→Fragment scope), component dependencies на границах фич-модулей, где важны изоляция и скорость. `@Module(subcomponents=...)` предпочтительнее ручного factory-метода, т.к. позволяет Dagger не генерировать неиспользуемый субкомпонент.

Почему это важно. Один из самых тонких вопросов по Dagger. Ответ про «multibindings не ходят через component dependencies» и про влияние на параллельную сборку — почти безошибочный маркер реального опыта масштабирования DI.

Вопрос 5.3 — Что такое assisted injection и multibinding, и зачем Hilt?

Junior. Multibinding собирает несколько объектов в Set/Map. Assisted — когда часть параметров передаём вручную. Hilt — это упрощённый Dagger.

Middle. `@IntoMap`/`@IntoSet` собирают вклады из разных модулей — классика для `Map<Class, ViewModel>` или `Map<Class, SubcomponentFactory>`. Assisted injection (`@AssistedInject`, `@Assisted`, `@AssistedFactory`) — когда объекту нужны и graph-зависимости, и рантайм-параметры (например, id из аргументов экрана). Hilt убирает бойлерплейт компонентов и даёт готовую иерархию (`@HiltViewModel`, `@AndroidEntryPoint`).

Senior. Multibinding — основа плагинных и масштабируемых архитектур: каждая фича вкладывает свой субкомпонент/ViewModel-фабрику в общую Map, и core-модуль не знает про фичи (инверсия). Assisted решает реальную боль — раньше передавали runtime-параметры через сеттеры или factory вручную, теперь типобезопасно. Про Hilt — это явный trade-off: он навязывает фиксированный набор компонентов (Singleton/Activity/Fragment/View/ViewModel) и центральную агрегацию модулей (всё генерируется вместе → нет выигрыша параллельной сборки, который дают component dependencies). Поэтому при кастомной Fragment-навигации и нестандартных scope'ах ручной Dagger даёт контроль, которого Hilt не даёт; Hilt выигрывает на типовых проектах скоростью разработки. Знать оба и осознанно выбирать — senior-уровень.

Почему это важно. Проверяет понимание DI как инструмента масштабирования, а не «магии аннотаций». Осознанный выбор Dagger vs Hilt по trade-off'ам, а не по моде — то, что отличает архитектора.

Тема 6. Room & DataStore

Теория

Room генерирует код через KSP (с Room 2.7+ KMP-совместим, по умолчанию генерирует Kotlin при обработке через KSP; Room 3.0 — KMP-first, только Kotlin, требует KSP и suspend/Flow для DAO, запрещает блокирующие DAO-методы). Миграции: auto-migrations (`@AutoMigration`), декларативно для простых изменений, с `AutoMigrationSpec` для переименований/сложных) против manual `Migration` (ручной SQL для нетривиальных трансформаций данных). `exportSchema=true` обязателен для тестируемых миграций. Room возвращает `Flow<T>` из DAO и перезапустит при изменении ЗАТРОНУТЫХ таблиц (через `InvalidationTracker`, отслеживающий запись по таблицам, не по строкам) — поэтому даже несвязанный апдейт той же таблицы триггерит перезапуск. `@Transaction` обеспечивает атомарность и консистентность связанных чтений (`@Relation`). Threading: блокирующие DAO-методы нельзя на main; suspend/Flow Room сам уводит на свой Executor. DataStore (Preferences vs Proto) — асинхронная замена SharedPreferences на coroutines/Flow с транзакционным `updateData()` (атомарный read-modify-write); Preferences без схемы (key-value), Proto — типобезопасный со схемой protobuf. DataStore не поддерживает частичные апдейты и referential integrity — для сложных данных это Room.

Вопрос 6.1 — auto-migration против manual, и где грабли?

Junior. Auto-migration Room делает сам, manual — пишем SQL руками. Надо повышать version.

Middle. Простые изменения (новая колонка, новая таблица) — `@AutoMigration(from, to)`. Переименования колонок/таблиц и сложные трансформации — manual `Migration` с SQL или `AutoMigrationSpec`. Нужен `exportSchema=true` и тесты миграций через `MigrationTestHelper`. Без миграции и с `fallbackToDestructiveMigration` данные теряются.

Senior. Auto-migrations покрывают структурные изменения, выводимые из diff схемы, но не могут угадать семантику (переименование = drop+add для Room, если не подсказать `@RenameColumn`). Любая трансформация ДАННЫХ (сплит колонки, изменение типа с конвертацией, бэкфилл) — только manual. Критично: миграции тестируются на реальных экспортированных схемах (`MigrationTestHelper`), схемы в test resources — Room Gradle Plugin их добавляет автоматически, иначе прод-краш на апдейте. `fallbackToDestructiveMigration` допустим только для кэшей, не для пользовательских данных. Грабли: миграции должны быть детерминированы и идемпотентны по цепочке (1→2→3 и 1→3 дают одинаковый результат); забытый `exportSchema` лишает возможности тестировать; в KMP/Room 2.7+ переход на SQLiteDriver меняет API миграций.

Почему это важно. Миграции — место, где баг = потеря данных пользователя или краш на старте после обновления. Ответ про тестирование на экспортированных схемах и про опасность destructive fallback — маркер ответственности за прод.

Вопрос 6.2 — Как Room эмитит Flow при изменениях и какие из этого следствия?

Junior. DAO возвращает Flow, и он обновляется, когда данные в таблице меняются.

Middle. Room через InvalidationTracker следит за таблицами; при записи в затронутую таблицу Flow переэмитит результат запроса. Отслеживание по таблице, а не по строке, поэтому несвязанный апдейт той же таблицы тоже триггерит переэмит. Flow от Room уже работает на фоновом Executor.

Senior. Гранулярность инвалидации — табличная, не строчная и не по WHERE-условию: запись в любую строку отслеживаемой таблицы переэмитит ВСЕ Flow-запросы к ней, даже если их результат не изменился. Следствия: на горячих таблицах это лишние переэмисы и recomposition — лечится `distinctUntilChanged()` на выходе и/или сужением запросов. Для `@Relation`-запросов несколько таблиц = инвалидация от любой из них. `@Transaction` нужен не только для записи, но и для согласованного многотабличного ЧТЕНИЯ, иначе между подзапросами данные «уедут». Поскольку StateFlow в UI дедуплит по equals, лишние переэмисы из Room часто гасятся уже на уровне UI-state, но лучше резать раньше. Threading: блокирующие методы на main кидают исключение by design; suspend/Flow безопасны.

Почему это важно. Многие думают, что Flow от Room «умный» и эмитит только при реальном изменении результата. Понимание табличной гранулярности объясняет необъяснимые recomposition и лишнюю работу — это глубина, видимая только в проде.

Вопрос 6.3 — DataStore Preferences vs Proto и атомарность?

Junior. DataStore заменяет SharedPreferences, работает асинхронно. Preferences — ключ-значение, Proto — типизированный.

Middle. Preferences DataStore — key-value без схемы, Proto — со схемой protobuf и типобезопасностью. Оба на coroutines/Flow, не блокируют main (в отличие от SharedPreferences с его синхронным apply/commit и риском ANR на fsync). `edit{}`/`updateData{}` атомарны. Для сложных/больших данных и частичных апдейтов — Room, не DataStore.

Senior. Главное преимущество над SharedPreferences — транзакционный атомарный `updateData()` (read-modify-write в одной suspend-операции, завершается после фактической записи на диск) и отсутствие синхронного main-thread I/O. Выбор Preferences vs Proto — это типобезопасность и эволюция схемы против простоты: Proto ловит ошибки типов в compile-time и корректно версионизируется, но требует .proto и сериализатора; Preferences проще, но `Any?`-подобная нетипизированность. Грабли: DataStore — это один файл и один экземпляр на файл (создание двух DataStore на один файл = краш/коррупция); нет частичных апдейтов (вся структура переписывается) и нет referential integrity — поэтому это для настроек/флагов, а не для коллекций. Нет встроенного шифрования — для токенов нужен ручной encrypt или сторонняя обёртка (например, encrypted-datastore). Чтение через Flow надо защищать `catch` для `IOException` (коррупция файла).

Почему это важно. Выбор хранилища — частый архитектурный вопрос. Ответ про атомарность `updateData`, про «один файл — один экземпляр» и про отсутствие шифрования показывает, что человек реально эксплуатировал DataStore, а не прочитал сравнительную табличку.

Тема 7. Retrofit / OkHttp

Теория

OkHttp различает два типа `interceptor`'ов. `Application interceptor` (`addInterceptor`): вызывается один раз на логический запрос, видит исходный `intent` приложения, выполняется даже при отдаче из кэша, может `short-circuit`'нуть (не вызывать `proceed`), не видит редиректы/ретраи. `Network interceptor` (`addNetworkInterceptor`): ближе к проводу, видит реальные байты/редиректы/ретраи и заголовки, добавленные OkHttp (`gzip`, `If-None-Match`), имеет ненулевой `Connection`, НЕ вызывается при отдаче из кэша (304). Кэш OkHttp слушает заголовки на `network`-уровне, поэтому форсить хранение лучше `network interceptor`'ом, а отдавать кэш офлайн — `application interceptor`'ом. Retrofit интегрируется с `coroutines` через `suspend`-функции (под капотом `Call.enqueue` + `Continuation`); возвращать можно `T`, `Response<T>` (для доступа к коду/заголовкам) или `Result`. `Converter factory` (Kotlin Serialization / Moshi) маппит тело. `Non-2xx`: `suspend fun(): T` бросает `HttpException` на 4xx/5xx; `Response<T>` — нет, надо проверять `isSuccessful`. Таймауты (`connect/read/write/call`) задаются на `OkHttpClient`.

Вопрос 7.1 — Чем отличаются `application` и `network interceptor`?

Junior. `Application interceptor` добавляется через `addInterceptor`, `network` — через `addNetworkInterceptor`. Первый выше, второй ниже.

Middle. `Application interceptor` вызывается один раз, видит исходный запрос, срабатывает даже на кэше — туда кладут `auth`-заголовки, логирование верхнего уровня. `Network interceptor` видит редиректы и ретраи (может вызваться несколько раз), не срабатывает на отдаче из кэша — туда кладут низкоуровневое логирование и переписывание `Cache-Control`.

Senior. Разница в позиции в цепочке и в семантике вызова. `Application`: один вызов на логический запрос, видит `intent` приложения до редиректов, ВСЕГДА вызывается (в т.ч. при `cache hit`), может `short-circuit` и ретраить с несколькими `proceed`. `Auth`-заголовки и токены — именно сюда, потому что они нужны независимо от кэша и редиректов. `Network`: видит реальную сетевую транзакцию, нон-нулл `Connection` (можно узнать `TLS/IP`), видит `OkHttp`-инъекции (`gzip`, `If-None-Match`) и НЕ вызывается на `304` из кэша — отсюда классический баг с `inspector`'ами (`Chucker`), которые «не видят» `cache hit` на `app`-уровне или дублируют запрос на `network`-уровне. Для кэширования: форсить запись ответа (переписать серверный `Cache-Control`) — `network interceptor`; форсить чтение из кэша офлайн (`CacheControl.FORCE_CACHE`) — `application interceptor`, т.к. он работает и без сети. `Auth`-рефреш лучше через `Authenticator`, а не `interceptor`, т.к. он реагирует на `401` реактивно.

Почему это важно. Это каноничный `senior`-вопрос по сети. Ответ про «`network` не вызывается на `304`» и про правильное место для `auth/cache` — прямой индикатор, что человек настраивал кэш и отлаживал сетевой слой, а не только дёргал `endpoint`'ы.

Вопрос 7.2 — Как обрабатывать ошибки и заворачивать в `sealed Result`?

Junior. Оборачиваем вызов в `try/catch`, ловим `Exception`, возвращаем успех или ошибку.

Middle. `suspend fun(): T` бросает `HttpException` на `non-2xx` и `IOException` на сети — ловим оба. Лучше вернуть `Response<T>` и проверить `isSuccessful`/`code()`, либо завернуть в `sealed class Result { Success, Error(...) }`. Парсинг тела ошибки — через `response.errorBody()`.

Senior. Зрелый подход — единый `sealed`-тип результата (`Success`/`HttpError(code, body)`/`NetworkError`/`UnknownError`) и общая `runCatching`-обёртка/`CallAdapter`, разделяющая транспортные ошибки (`IOException` — нет сети, таймаут), протокольные (`HttpException` — есть код и `errorBody` для типизированного парсинга) и сериализационные. Ключевое `senior`-различие: НЕ глотать `CancellationException` в общем `catch` (иначе ломается `structured concurrency`) — её нужно пробрасывать. Маппинг доменных ошибок (`401`→разлогин, `409`→конфликт) делаем в одном месте, а не в каждом репозитории. `Response<T>` против `T`: первый не бросает на `non-2xx` и даёт доступ к коду/заголовкам, но требует ручной проверки; голый `T` лаконичнее, но прячет коды в исключениях. Таймауты разделяем: `callTimeout` ограничивает весь запрос включая ретрай, `read/connect/write` — фазы.

Почему это важно. Обработка ошибок — то, что отличает прод-код от демо. Ответ про непроглатывание `CancellationException` и про разделение типов ошибок показывает, писал ли человек устойчивый сетевой слой, а не `happy-path`.

Теория

В кастомной архитектуре экраны — это `Fragment`'ы, hostящие `Compose` через `ComposeView` в `onCreateView`, а `back-stack` управляется собственным `attacher`'ом (`pushToStack`/`popStack`/`replace`) поверх `FragmentManager`-транзакций. Критично выставлять `ComposeView.setViewCompositionStrategy(DisposeOnViewTreeLifecycleDestroyed)`: дефолтная стратегия диспозит композицию при `detach` от окна, что во `Fragment`'ах ломает сохранение `state` (например, `scroll position`) во время транзакций/анимаций, когда `view` ещё не уничтожен, но уже `detached`. `DisposeOnViewTreeLifecycleDestroyed` привязывает диспоз к жизненному циклу `view` фрагмента (1:1 с `ViewTreeLifecycleOwner`). Lifecycle `Fragment`'а с `Compose`: `viewLifecycleOwner` для подписок (не сам `Fragment`), `rememberSaveable` сохраняет UI-state через `SavedStateRegistry` фрагмента. `Navigation Compose` (`NavController`) — альтернатива: единый `Activity`, `composable-destination`'ы, типобезопасные маршруты, но требует перехода всех экранов на `composable` и иной модели `back-stack/scoping`.

Вопрос 8.1 — Зачем ComposeView и какая ViewCompositionStrategy внутри Fragment?

Junior. `ComposeView` позволяет положить `Compose` внутрь `Fragment` через `setContent`. Стратегию выставлять не обязательно, есть дефолтная.

Middle. В `onCreateView` создаём `ComposeView`, ставим `setViewCompositionStrategy(DisposeOnViewTreeLifecycleDestroyed)` и `setContent { }`. Без правильной стратегии при возврате по `back-stack` или анимации теряется `state` (`scroll`, ввод), потому что дефолт диспозит композицию при `detach` от окна.

Senior. Дефолтная стратегия (`DisposeOnDetachedFromWindowOrReleasedFromPool`) диспозит композицию при `detach` от окна — а во `Fragment`'е `view` может быть `detached`, но `lifecycle` ещё жив (транзакции, анимации перехода, `add-to-backstack`). Это даёт два бага: преждевременный диспоз до `onDestroyView` (где `Fragment` сохраняет `state`) → потеря `Compose` UI-state; и визуальные артефакты, когда `composable` «висит» во время перехода. `DisposeOnViewTreeLifecycleDestroyed` привязывает диспоз к `viewLifecycleOwner` (1:1 с `view` фрагмента) — композиция живёт ровно столько, сколько `view`. Тонкость: подписки на `Flow` внутри `Fragment`'а используют `viewLifecycleOwner.lifecycleScope` и `repeatOnLifecycle`, а не сам `Fragment` (его `lifecycle` переживает `view` → утечка). Для нескольких `ComposeView` в одном фрагменте важны уникальные `id`, чтобы `SavedStateRegistry` корректно сохранял `state` каждого.

Почему это важно. Это конкретная боль интеропа, на которой ловятся даже опытные. Знание, ПОЧЕМУ нужна именно `DisposeOnViewTreeLifecycleDestroyed` (`detach` ≠ `destroy`), и разница `viewLifecycleOwner` vs `Fragment` — точный маркер реального опыта `Compose`-внутри-`Fragment`.

Вопрос 8.2 — Кастомный Fragment back-stack против Navigation Compose: trade-offs?

Junior. Своя навигация даёт контроль через `push/pop`, `Navigation Compose` — стандартная и проще, но менее гибкая.

Middle. Кастомный `attacher` поверх `FragmentManager` даёт явный контроль над стеком и переиспользует зрелый `Fragment lifecycle` (`ViewModel scoping`, `SavedState`). `Navigation Compose` убирает `Fragment`'ы, даёт типобезопасные маршруты и `composable-destination`'ы, но требует мигрировать все экраны на `composable` и иначе управлять `scoping/back-stack`.

Senior. Это выбор между зрелостью/контролем и идиоматичностью. Кастомная `Fragment`-навигация опирается на годами обкатанный `FragmentManager`: гарантированное сохранение состояния через `SavedStateRegistry`, `ViewModel scoping` к `Fragment`, предсказуемый `lifecycle`, и полный контроль над транзакциями (анимации, `multi-stack`, переиспользование). Минусы — бойлерплейт, ручное управление стеком, риск рассинхрона `attacher`'а с `FragmentManager`, и интероп-нюансы `ComposeView`. `Navigation Compose` даёт типобезопасные маршруты (с `serialization`), декларативный граф и убирает `Fragment`-прослойку, но: иная модель `scoping` (`ViewModel` к `back-stack entry`), исторически были вопросы к сохранению сложного `state` и к анимациям, и переход требует, чтобы `BCE destination`'ы стали `composable`. Прагматично: в большом `legacy`-проекте с `Fragment`-навигацией и постепенной миграцией на `Compose` кастомный `attacher` оправдан; для нового `Compose-first` проекта `Navigation Compose` идиоматичнее. «It depends» здесь честный.

Почему это важно. Проверяет архитектурную трезвость. `Junior` хочет «новое и модное»; `senior` взвешивает стоимость миграции, зрелость `scoping/SavedState` и контекст проекта, и не презирает `Fragment`'ы за то, что они «старые».

Тема 9. Стратегия тестирования

Теория

`JUnit5` даёт `@BeforeEach/@AfterEach`, `@Nested`, `@ParameterizedTest` и `extensions` (вместо `Rule`'ов `JUnit4`) — например, кастомный `TestDispatcher`-extension для подмены `Main`. `MockK` — `Kotlin-first`: `coEvery`/`coVerify` для `suspend`, `relaxed`/`relaxedUnitFun` для авто-стабов, `slot`/`CapturingSlot` для захвата аргументов. `Coroutines`: `runTest` (заменял `runBlockingTest`) исполняет тело на `TestScope`, пропускает `delay` (виртуальное время) и ждёт завершения дочерних `coroutine`. По умолчанию `runTest` использует `StandardTestDispatcher` — новые `coroutine` ставятся в очередь и НЕ исполняются `eagerly`, нужен `runCurrent()`/`advanceUntilIdle()`/`advanceTimeBy()`; `UnconfinedTestDispatcher` запускает их `eagerly` (удобно для `Flow`, но порядок менее явный). `Main` подменяется `Dispatchers.setMain(StandardTestDispatcher(...))`. Все `TestDispatcher`'ы должны делить один `TestCoroutineScheduler`. `Flow` тестируем `Turbine` (`awaitItem`/`awaitComplete`/`awaitError`/`ensureAllEventsConsumed`). `Compose UI`: `createComposeRule`, семантика, `onNodeWithText`, синхронизация по `idle`.

Вопрос 9.1 — `StandardTestDispatcher` против `UnconfinedTestDispatcher`?

Junior. Оба для тестов coroutines. Standard — по умолчанию в runTest, Unconfined запускает сразу.

Middle. StandardTestDispatcher ставит новые coroutine в очередь — нужно вручную продвигать (advanceUntilIdle), (runCurrent), что даёт точный контроль порядка. UnconfinedTestDispatcher исполняет coroutine eagerly до первого suspend — удобно, когда нужно, чтобы collect/launch стартовал сразу (тесты Flow/StateFlow). runTest по умолчанию Standard.

Senior. Это контроль времени против простоты. Standard (дефолт runTest) не запускает дочерние coroutine eagerly — они в очереди scheduler'a; это позволяет выставить precondition'ы ДО исполнения и детерминированно проверять порядок, но требует явного runCurrent/advanceUntilIdle, и наивный тест «launch collect; меняю StateFlow; проверяю» провалится, т.к. коллектор ещё не стартовал. Unconfined запускает eagerly до первой точки suspend (только на верхнем уровне — для вложенных launch/async гарантий порядка нет), что упрощает тесты горячих потоков, но прячет проблемы с порядком. Критично: все TestDispatcher'ы в тесте делят один scheduler (через testScheduler), иначе виртуальное время рассинхронится; withContext(Dispatchers.IO) в коде ломает скийп delay — поэтому диспетчеры инжектируем. Для StateFlow-тестов чаще Unconfined + Turbine; для проверки последовательности шагов — Standard.

Почему это важно. Самая частая причина flaky/«непонятно почему падает» coroutine-тестов — непонимание, что Standard не запускает coroutine сами. Ответ про общий scheduler и про инъекцию диспетчеров — маркер того, что человек реально писал стабильные async-тесты.

Вопрос 9.2 — Зачем Turbine и MockK coEvery/coVerify/slot?

Junior. Turbine собирает эмиссии Flow, MockK мокает, coEvery для suspend-функций.

Middle. Turbine: flow.test { awaitItem(); awaitComplete() } — удобнее ручного сбора в список, проверяет порядок и завершение. MockK: coEvery { repo.load() } returns x стабит suspend; coVerify { } проверяет вызов suspend; slot/CapturingSlot ловит переданный аргумент для ассертов. relaxed мок не требует стабить каждый вызов.

Senior. Turbine незаменим для горячих потоков (StateFlow/SharedFlow), потому что форсит дисциплину: ensureAllEventsConsumed падает, если осталась непроверенная эмиссия — это ловит лишние/потерянные значения, чего ручной toList() (который к тому же не завершится на бесконечном hot-потоке) не умеет; помним про начальное значение StateFlow и его conflation/дедуп по equals. MockK: relaxed ускоряет, но прячет незастабленые вызовы — для критичных зависимостей лучше явные стабы; coVerify(exactly = n)/coVerifySequence проверяют не только факт, но и количество/порядок; slot + answers { } позволяют возвращать значение, зависящее от аргумента. Грабль: verify на suspend (нужен coVerify), и мок объектов с финальными классами (MockK умеет, но требует осознанности). Тесты должны проверять поведение (последовательность состояний), а не реализацию.

Почему это важно. Инструменты — это просто; вопрос проверяет вкус: использует ли человек `ensureAllEventsConsumed` и `coVerifySequence` для надёжности, или пишет хрупкие тесты, проходящие случайно.

Вопрос 9.3 — Как тестировать Compose UI и восстановление state?

Junior. `createComposeRule`, `onNodeWithText("...").performClick()`, проверяем `assertIsDisplayed`.

Middle. Compose-тесты используют semantics tree: `onNodeWithText/Tag`, действия (`performClick`), ассерты (`assertExists`). Синхронизация автоматическая по `idle`, но при отключённой авто-синхронизации нужен `waitForIdle`/`mainClock`. Для восстановления state — `StateRestorationTester`.

Senior. Compose UI-тесты опираются на semantics, поэтому тестопригодность закладывается через `Modifier.testTag` и осмысленные content descriptions. Ключевое для надёжности — управление часами: при анимациях/`LaunchedEffect` авто-синхронизация может не дожидаться, тогда `mainClock.autoAdvance = false` и ручное продвижение. `StateRestorationTester` проверяет, что `rememberSaveable`-state корректно переживает recreation — без этого легко пропустить баг, где state теряется при process death. Тонкость `rememberSaveable`: он восстанавливает значение при ВОЗВРАТЕ по back-stack как при первом входе (re-enter composition), что не то же самое, что «запомнить последнее» — это надо тестировать осознанно. Для интеграции с coroutine-UI инжектируем `TestDispatcher` в `Main`. Тестируем поведение пользователя, а не внутренние state-объекты.

Почему это важно. Compose-тесты легко сделать flaky из-за асинхронности и анимаций. Знание про управление `mainClock` и `StateRestorationTester` показывает, что человек писал устойчивые UI-тесты, а не только «нажми кнопку — проверь текст».

Тема 10. Build & Performance

Теория

R8 (дефолтный shrinker, заменил ProGuard) делает tree shaking (удаление недостижимого кода), оптимизацию (инлайнинг, class merging, удаление мёртвых веток), обфускацию (укорачивание имён) и desugaring — за один проход с dexing. Full mode — дефолт с AGP 8.0: агрессивнее (меняет видимость для инлайнинга, не сохраняет default-конструкторы и аннотации без явных keep). С AGP 8.12+ — optimized resource shrinking (код и ресурсы в одном графе). `proguard-android-optimize.txt` обязателен вместо устаревшего `proguard-android.txt` (последний не включает оптимизации; AGP 9.0 его не поддерживает — Disney+, по данным Android Developers Blog, после замены `proguard-android.txt` на `proguard-android-optimize.txt` получил «30% faster app startup and 25% fewer user-perceived ANRs»). Keep-правила должны быть узкими; `-keep class **/{-keep com.app.**}` убивают весь смысл оптимизации. Baseline Profiles предкомпилируют горячие пути (AOT) для critical user journeys, убирая JIT-затраты на первых запусках; генерируются через Macrobenchmark + BaselineProfileRule, измеряются через `CompilationMode.None()` vs `Partial(BaselineProfileMode.Require)` со `StartupTimingMetric`/`FrameTimingMetric`. Модуляризация ускоряет сборку (параллелизм, инкрементальность) и даёт инверсию через `api/implementation`: `implementation` не протекает в потребителей (быстрее перекомпиляция), `api` — протекает (для транзитивного API). Compose compiler с Kotlin 2.0+ в репозитории Kotlin, версия = версии Kotlin, настраивается `composeCompiler {}` (метрики через `reportsDestination`); strong skipping — дефолт с 2.0.20. [Google](#) [Google](#)

Вопрос 10.1 — Что делает R8 full mode и как писать keep-правила?

Junior. R8 уменьшает и обфусцирует APK. Keep-правила не дают удалить нужные классы, например модели для Gson.

Middle. R8 делает shrinking (tree shaking), оптимизацию (инлайнинг) и обфускацию за один проход. Full mode (дефолт с AGP 8.0) агрессивнее, поэтому может ломать reflection — нужны keep-правила. Правила должны быть узкими, а не `-keep com.app.**`. Для отладки стектрейсов сохраняем mapping.txt.

Senior. R8 — это static analysis, поэтому всё динамическое (reflection, JNI, сериализация по полям) невидимо и требует keep. Full mode дополнительно меняет видимость (`private`→`public` для инлайнинга), не сохраняет default-конструкторы (ломает `newInstance()` по рефлексии) и не держит аннотации без явного keep — типичные краши после включения. Дисциплина: правила максимально узкие, предпочтительно через общий базовый класс/интерфейс или `@Keep`-аннотацию рядом с кодом; модификаторы `allowshrinking`/`allowobfuscation` возвращают R8 свободу даже на вынужденных keep. Анти-паттерны: глобальные `-dontobfuscate`/`-donoptimize` в релизе, `-keep` на весь пакет приложения (платишь за R8, не получая выгоды), ручные keep для Activity/Service (уже в дефолтном файле). Внедрять инкрементально: начать с tree shaking, затем включить full mode, тестировать релизный билд на реальном устройстве. Эффект реален: по данным Android Developers Blog (ноябрь 2025), Reddit после включения R8 в full mode «observed a 40% faster cold startup, a 30% reduction in 'Application Not Responding' (ANR) errors, a 25% improvement in frame rendering, and a 14% reduction in app size», а полностью оптимизированная сборка (R8 + Baseline + Startup Profiles) «started 55% faster and users could begin to browse 18% sooner» с «a two-thirds reduction in JIT compilation occurrences». [Google](#) [Googblogs](#)

Почему это важно. R8 — частый источник «работает в debug, падает в release». Ответ про невидимость reflection, про изменение видимости в full mode и про узкие правила/allowobfuscation показывает, что человек настраивал и дебажил релизную минификацию, а не копировал чужой proguard-файл.

Вопрос 10.2 — Как работают Baseline Profiles и как измерять эффект?

Junior. Baseline Profiles ускоряют запуск, предкомпилируя код. Генерируются через бенчмарк.

Middle. ART на установке не компилирует всё в нативный код (JIT на лету), поэтому первые запуски медленные. Baseline Profile перечисляет горячие методы/классы для AOT-компиляции при установке. Генерируем через Macrobenchmark + BaselineProfileRule, прогоняя critical user journeys; измеряем StartupTimingMetric с CompilationMode.None() vs Partial(Require).

Senior. Профиль — это список правил компиляции для критичных путей (старт, скролл ключевых экранов), который ART применяет при установке/обновлении, убирая JIT-разогрев и interpretation на первых сессиях. Генерация: Macrobenchmark-тест с UiAutomator проигрывает реальные journey, собирая профиль; качество профиля = качество сценария, поэтому покрывают именно важные пути и регенерируют при заметных изменениях навигации. Измерение: сравнивают CompilationMode.None() (как без профиля) и Partial(BaselineProfileMode.Require); метрики — StartupTimingMetric (TTID и важнее TTFD — время до полностью интерактивного экрана) и FrameTimingMetric для jank. Прогонять на физическом устройстве или Gradle Managed Device (воспроизводимость), не на каждом PR, а nightly, с трендом в CI по JSON-выводу. Связка с R8: при включённом R8 рекомендуют ещё Startup Profiles. Эффект конкретен: по документации Android, Baseline Profiles «improve code execution speed by about 30% from the first launch by avoiding interpretation and just-in-time (JIT) compilation steps», а в Google codelab «the difference between compilation modes for timeToFullDisplay is 180ms, which is ~28% improvement by just having a Baseline Profile». Reddit после апгрейда на Compose-релиз сентября 2024 получил «cold start times improved by 13%... app start times improved by up to 40%» на low/mid-tier устройствах и «a 23% reduction in frozen frames» в home feed. Android Developers + 2

Почему это важно. Перф-инициатива, дающая реальный ROI почти бесплатно. Ответ про TTID vs TTFD, про Gradle Managed Devices и про регенерацию профилей при изменении journey — маркер того, что человек реально гонял Macrobenchmark, а не читал про него.

Вопрос 10.3 — Зачем модуляризация и в чём разница api/implementation?

Junior. Модули ускоряют сборку и разделяют код. implementation скрывает зависимость, api — пробрасывает дальше.

Middle. Модуляризация даёт параллельную и инкрементальную сборку (меняешь один модуль — пересобирается он, а не всё) и разделение ответственности. `implementation` не протекает в потребителей — изменение его API не триггерит их перекомпиляцию; `api` протекает транзитивно, нужен когда тип зависимости — часть публичного API модуля. По умолчанию всегда `implementation`.

Senior. Модуляризация — это и скорость сборки (параллелизм Gradle, инкрементальность, меньший classpath на модуль), и архитектура (инверсия через api/impl-разделение, изоляция фич, чёткие границы). `implementation` критичен для скорости: он создаёт «стену компиляции» — потребители не видят транзитивную зависимость, поэтому её изменение не каскадит перекомпиляцию; `api` намеренно протекает и используется, только когда тип реально часть публичного контракта (возвращается/принимается в сигнатурах). Злоупотребление `api` сводит на нет выигрыш модуляризации, превращая граф в монолит по времени сборки. Связка с DI: интерфейсы в api-модуле, реализации с `@Binds` в impl-модуле дают и инверсию зависимостей, и параллельную сборку (перекликается с component dependencies в Dagger). Version Catalogs (`libs.versions.toml`) централизуют версии для консистентности между модулями. Метрики Compose-компилятора и build scan помогают находить узкие места сборки.

Почему это важно. Скорость сборки — ежедневная боль больших команд. Ответ про «implementation = стена компиляции» и про осознанное ограничение api показывает, оптимизировал ли человек реальный build time, а не просто «разбил на модули».

Тема 11. Process & app lifecycle, background work

Теория

Три типа старта: cold (процесс создаётся с нуля — самый дорогой, цель Baseline Profiles), warm (процесс жив, Activity пересоздаётся), hot (всё в памяти, на передний план). Configuration change (поворот) уничтожает и пересоздаёт Activity, но процесс жив — ViewModel переживает это (хранится вне lifecycle Activity). Process death — система убивает процесс в фоне под нехватку памяти; при возврате Activity пересоздаётся в «чистом» состоянии, ViewModel НЕ переживает.

`SavedStateHandle` (для бизнес-логики в ViewModel) и `rememberSaveable` (для UI-логики в Compose) переживают и configuration change, и process death через `Bundle`/`SavedStateRegistry` — но только МАЛЫЕ данные (лимит Binder-транзакции ~1MB), для тяжёлого — local persistence (Room/DataStore), а в Bundle класть лишь ключ/id. Saved state привязан к task stack: при force-stop/свайпе из recents/ребуте он теряется. WorkManager — для гарантированной отложенной фоновой работы, переживающей закрытие app и ребут: constraints (сеть/зарядка/idle), chaining (`beginWith().then()`), uniqueness (`enqueueUniqueWork` + `ExistingWorkPolicy` KEEP/REPLACE/APPEND/APPEND_OR_REPLACE), retry (exponential/linear backoff, мин. 10с), expedited work (с WorkManager 2.7.0), периодика (мин. интервал 15 мин). WorkManager — НЕ для in-process работы, которую можно потерять (это coroutines/scope).

Вопрос 11.1 — Чем configuration change отличается от process death и что переживает что?

Junior. Поворот пересоздаёт Activity, ViewModel переживает поворот. Process death — система убивает приложение, тогда нужен SavedStateHandle.

Middle. При configuration change процесс жив, Activity пересоздаётся, ViewModel сохраняется (он привязан не к Activity-инстансу). При process death процесс убит — ViewModel теряется, восстановить помогает `SavedStateHandle`/`rememberSaveable` (через Bundle). В Bundle — только маленькие данные; тяжёлое перезагружаем из repository по сохранённому id.

Senior. Это два разных события с разной выживаемостью. Configuration change: процесс и ViewModel живы (ViewModel хранится в `ViewModelStore`, переживающем пересоздание Activity), теряется только то, что в самой Activity/view. Process death: всё в памяти уничтожено, при возврате система пересоздаёт Activity и хочет «бесшовности» — её обеспечивают `SavedStateHandle` (бизнес-state в ViewModel) и `rememberSaveable` (UI-state), сериализуемые в `Bundle`. Границы: Bundle ограничен Binder-транзакцией (~1MB) → класть только минимум (id, scroll position, ввод), остальное восстанавливать из Room/DataStore; saved state привязан к task stack и НЕ переживает force-stop/свайп из recents/ребут (там — только persistent storage). Тестировать: «Don't keep activities» + `adb shell am kill`. Тонкость: `rememberSaveable` при возврате по back-stack восстанавливает как при первом входе, что не равно «помнить последнее».

Почему это важно. Process death — не edge case, а часть нормальной жизни Android-процесса; многие тестируют только поворот и ловят баги «чистого экрана» в проде. Ответ про лимит Bundle, про привязку saved state к task stack и про разницу хранилищ — маркер зрелости.

Вопрос 11.2 — WorkManager: constraints, chaining, uniqueness, retry — когда и как?

Junior. WorkManager запускает фоновые задачи с условиями (сеть, зарядка), умеет цепочки и повторы. `enqueueUniqueWork` чтобы не дублировать.

Middle. Constraints (`NetworkType`, `charging`, `battery-not-low`, `idle`) откладывают запуск до выполнения условий. Chaining: `beginWith(a).then(b).enqueue()`, выход одной задачи → вход следующей; при падении звена цепочка останавливается. Uniqueness: `enqueueUniqueWork(name, policy, request)` с `KEEP`/`REPLACE`/`APPEND`/`APPEND_OR_REPLACE`. Retry: `Result.retry()` + `backoff` (`exponential/linear`, мин. 10с). Периодика — мин. 15 мин.

Senior. WorkManager — для гарантированной отложенной работы, переживающей закрытие app и ребут (под капотом `JobScheduler/AlarmManager`); не для немедленной `in-process` работы (это `coroutines` в правильном `scope`). Constraints — минимально необходимые: переусложнённый набор (`charging` + `idle` + `unmetered`) может не выполниться никогда.

Uniqueness-политики осмысленно: для периодической синхронизации `KEEP` (не плодить дубликаты), для «последний выигрывает» — `REPLACE` (отменяет текущую), `APPEND` строит цепочку, но при падении/отмене `prerequisite` новая работа тоже падает — для независимости `APPEND_OR_REPLACE`. `Backoff exponential` для сетевых ретраев (минимум 10 секунд). `Expedited work` (с WorkManager 2.7.0) для важных коротких задач с квотами и `getForegroundInfo`/`fallback` `RUN_AS_NON_EXPEDITED`. Грабли: периодика игнорирует интервал <15 мин (тихо, без краша); `cooperative cancellation` в `onStopped()` (закрыть ресурсы при превышении 10 мин или снятии constraints); тестирование через `WorkManagerTestInitHelper`. Паттерн `offline-first`: записать в Room → `enqueue unique work` → синхронизировать при появлении сети.

Почему это важно. WorkManager часто применяют не туда (для немедленной работы) или ловят баги дублирования/тихого игнора интервала. Ответ про выбор политики `uniqueness` под сценарий, про `onStopped` и про «не для `in-process`» показывает реальную эксплуатацию, а не знание API.

Вопрос 11.3 — Как оптимизировать cold start?

Junior. Поменьше делать в `Application.onCreate` и в старте первой Activity.

Middle. Cold start — самый дорогой (создание процесса, инфляция, первый кадр). Убираем тяжёлую инициализацию из `Application.onCreate`, ленивая инициализация библиотек (App Startup), не блокируем `main`, откладываем некритичное. `Baseline Profiles` ускоряют первые запуски. Измеряем `Macrobenchmark` `StartupTimingMetric`.

Senior. Системно: (1) минимизировать `Application.onCreate` — синхронная инициализация SDK здесь напрямую бьёт по TTID; ленивая/фоновая инициализация (App Startup, отложенный DI-граф). (2) Baseline + Startup Profiles убирают JIT/interpretation на горячем пути старта. (3) R8 full mode + optimized resource shrinking уменьшают DEX и ускоряют загрузку классов. (4) Избегать тяжёлой работы до первого кадра, не делать синхронный I/O/БД на main. (5) Измерять TTID и особенно TTFD (полностью интерактивный экран) через Macrobenchmark на Gradle Managed Device, отслеживать тренд в CI. (6) Perfetto/трейсы для поиска, что именно тормозит до первого кадра. Цель — не «быстро по ощущениям», а измеримое TTFD с защитой от регрессий.

Почему это важно. Cold start — первое впечатление пользователя и метрика, коррелирующая с retention. Ответ, связывающий `Application.onCreate`, Baseline Profiles, R8 и измерение TTFD в единую стратегию, показывает системное перф-мышление, а не точечные советы.

Практические задачи (Kotlin)

Задача 1 — Кастомный Flow-оператор: debounce с leading edge

kotlin

```
// Зачем: проверяет понимание холодных Flow, flow{}/collect, корутинного времени,
// и того, как писать собственный оператор поверх примитивов.
// Раскрывает: Темы 1, 2 (структурированная конкурентность, отмена, операторы Flow).
fun <T> Flow<T>.debounceLeading(timeoutMillis: Long): Flow<T> = flow {
    var lastEmitTime = 0L
    collect { value ->
        val now = System.currentTimeMillis()
        if (now - lastEmitTime >= timeoutMillis) {
            lastEmitTime = now
            emit(value)           // эмитим первый (leading) элемент сразу
        }                       // последующие в окне — игнорируем
    }
}

// Senior-нюанс: для тестируемости время надо инжектировать (TimeSource), а не дёргать
// System.currentTimeMillis(), иначе runTest с виртуальным временем не проверит логику.
```

Задача 2 — retry с экспоненциальным backoff и jitter

kotlin

```
// Зачем: реальный сетевой паттерн; проверяет работу с suspend, отменой и ошибками.
// Раскрывает: Темы 1, 2, 7 (обработка ошибок, не глотать CancellationException).
suspend fun <T> retryWithBackoff(
    maxAttempts: Int = 3,
    initialDelayMs: Long = 500,
    factor: Double = 2.0,
    block: suspend () -> T
): T {
    var currentDelay = initialDelayMs
    repeat(maxAttempts - 1) {
        try {
            return block()
        } catch (e: CancellationException) {
            throw e // КРИТИЧНО: пробрасываем отмену, не ретраим
        } catch (e: Exception) {
            // лог через Timber.w(e)
        }
        delay(currentDelay + (0..100).random()) // jitter против "громового стада"
        currentDelay = (currentDelay * factor).toLong()
    }
    return block() // последняя попытка — без catch, ошибка наружу
}
```

Задача 3 — Чистый Reducer + тест (MVI)

kotlin

```
// Зачем: показывает, что Reducer — чистая синхронная функция, тривиально тестируемая.
// Раскрывает: Темы 4, 9 (FSM, table-driven тесты без моков и корутин).
data class State(val isLoading: Boolean = false, val items: List<String> = emptyList(), val error: String? = null)
sealed interface Effect {
    data object Loading : Effect
    data class Loaded(val items: List<String>) : Effect
    data class Failed(val message: String) : Effect
}
val reducer: (State, Effect) -> State = { state, effect ->
    when (effect) {
        Effect.Loading      -> state.copy(isLoading = true, error = null)
        is Effect.Loaded    -> state.copy(isLoading = false, items = effect.items)
        is Effect.Failed    -> state.copy(isLoading = false, error = effect.message)
    }
}
// Тест (JUnit5):
@Test fun `loaded effect fills items and clears loading`() {
    val result = reducer(State(isLoading = true), Effect.Loaded(listOf("a", "b")))
    assertEquals(State(isLoading = false, items = listOf("a", "b")), result)
}
```

Задача 4 — Ловушка recomposition и её фикс (Compose)

kotlin

```
// Зачем: демонстрирует deferred read через lambda-modifier – лучший способ убрать
// recomposition в горячем пути (скролл/анимация).
// Раскрывает: Тема 3 (фазы Compose, deferred reads, stability).
// ПЛОХО: чтение state в composition-фазе → recomposition на каждый кадр прокрутки
@Composable
fun BadHeader(scroll: ScrollState) {
    val offset = scroll.value // читаем в composition
    Text("Title", modifier = Modifier.offset(y = offset.dp)) // recomposition каждый пиксель
}
// ХОРОШО: чтение отложено в layout-фазу через lambda-modifier
@Composable
fun GoodHeader(scroll: ScrollState) {
    Text("Title", modifier = Modifier.offset { IntOffset(0, scroll.value) }) // skip composition
}
```

Задача 5 — StateFlow-кэширующий репозиторий

```
kotlin

// Зачем: каноничный паттерн кэша на горячем потоке с правильным SharingStarted.
// Раскрывает: Темы 2, 6 (stateIn, WhileSubscribed(5000), cold→hot, Room→Flow).
class UserRepository(
    private val dao: UserDao,
    private val scope: CoroutineScope,          // внешний scope, не на запрос!
) {
    // Свойство, а не функция: иначе на каждый вызов создается новый StateFlow.
    val users: StateFlow<List<User>> = dao.observeAll()    // Flow от Room
        .distinctUntilChanged()                          // режим лишние переземы (таблич. инвалидация)
        .stateIn(
            scope = scope,
            started = SharingStarted.WhileSubscribed(5_000), // переживает rotation
            initialValue = emptyList()
        )
}

// Senior-нюанс: WhileSubscribed(5000) держит upstream 5с после ухода подписчика;
// replayExpiration по умолчанию (Long.MAX_VALUE) не сбрасывает кэш — для большинства UI это ок.
```

Задача 6 — Конвертация RxJava-цепочки в Flow

kotlin

```
// Зачем: типовая миграция; проверяет понимание маппинга операторов И семантики потоков.
// Раскрывает: Темы 1, 2, 7 (switchMap→flatMapLatest, subscribeOn→flowOn, error handling).
// БЫЛО (RxJava):
// searchQueries
//     .debounce(300, MILLISECONDS)
//     .switchMap { q -> api.search(q).subscribeOn(Schedulers.io()) }
//     .observeOn(AndroidSchedulers.mainThread())
//     .subscribe({ render(it) }, { showError(it) })

// СТАЛО (Flow):
fun observeSearch(queries: Flow<String>): Flow<SearchResult> =
    queries
        .debounce(300) // оператор Flow, виртуальное время в тестах
        .flatMapLatest { q -> flow { emit(api.search(q)) } } // switchMap → flatMapLatest
        .flowOn(Dispatchers.IO) // upstream на IO (аналог subscribeOn)
        .catch { e -> emit(SearchResult.Error(e)) } // ловит ТОЛЬКО upstream-ошибки
// Сбор на UI: lifecycleScope + repeatOnLifecycle(STARTED) { collect { render(it) } }
// Нюанс: .catch не поймает исключение в render() (downstream) – это частая ошибка миграции.
```

Заметки об актуальности (2026) и осторожность с источниками

- **Strong skipping:** семантика `===` для `unstable` и `equals()` для `stable` подтверждена тремя первоисточниками (`developer.android.com`, `design-doc` JetBrains, статья Ben Trengrove). Вторичные блоги, утверждающие `equals()` для `unstable`-параметров, ошибочны.
- **Compose compiler:** в репозитории Kotlin с 2.0.0, версия плагина `org.jetbrains.kotlin.plugin.compose` совпадает с версией Kotlin.
- **Room 3.0** (анонс март 2026): KMP-first, только Kotlin, обязателен KSP, DAO — `suspend/Flow`, блокирующие методы запрещены. Это `breaking change` относительно Room 2.x; в проектах на Room 2.7+ многое уже применимо.
- **Числа по перфу** (Reddit, Disney+) — из официального Android Developers Blog и Google codelab; это конкретные кейсы, не гарантия одинакового результата на любом приложении — величина выигрыша зависит от кода.
- **`WhileSubscribed(5000)`**: «магическое» число — эвристика Google против рестарта `upstream` при `configuration change`, а не универсальная константа; для `one-shot` загрузок рассмотрите кастомные стратегии (`OnetimeWhileSubscribed`).